

Aprendizaje Supervisado

Clase 22: SVM y K-Nearest Neighbors

Dr. José Bavio

Departamento de Matemática
Universidad Nacional del Sur

2026

Contenido de la clase

- 1 Support Vector Machines
- 2 K-Nearest Neighbors
- 3 Comparación SVM vs KNN

Idea central de SVM

Problema: clasificar datos en dos clases ($y \in \{-1, +1\}$).

Hay muchos hiperplanos que separan las clases. ¿Cuál elegir?

Respuesta de SVM: el de máximo margen

Elegir el hiperplano que maximiza la **distancia** entre sí mismo y las observaciones más cercanas de cada clase.

Intuición geométrica

Es como trazar la línea divisoria entre dos territorios de forma que quede lo más lejos posible de ambos lados.

Cuanto más “espacio libre” hay a ambos lados, más confianza tenemos al clasificar un punto nuevo que cae cerca del borde.

Hiperplano y margen

El hiperplano se define como:

$$\mathcal{H} = \{x : w^T x + b = 0\}$$

Los **vectores de soporte** son las observaciones más cercanas al hiperplano: las únicas que determinan la solución. El resto de los datos son irrelevantes.

Coloquialmente

De todos los datos de entrenamiento, solo importan los que están “al borde”.

Si movés cualquier otro punto, el hiperplano no cambia.

Si movés un vector de soporte, sí cambia.

Margen = $2/\|w\|$. Maximizar el margen $\Leftrightarrow$ minimizar $\|w\|^2$.

Problema de optimización (datos separables)

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{sujeto a} \quad y_i(w^\top x_i + b) \geq 1 \quad \forall i$$

La restricción $y_i(w^\top x_i + b) \geq 1$ dice: cada observación debe estar al lado correcto del margen (no solo del hiperplano).

Limitación: datos no separables

Si las clases se solapan, ningún hiperplano satisface todas las restricciones. El margen duro no existe. Hay que relajar las restricciones.

SVM de margen blando

Se introducen **variables de holgura** $\xi_i \geq 0$ que permiten violaciones:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{sujeto a} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

El parámetro C controla el compromiso:

- C **grande**: penaliza mucho las violaciones $\Rightarrow$ margen más angosto, más riesgo de sobreajuste.
- C **pequeño**: tolera violaciones $\Rightarrow$ margen más amplio, más regularización.

Analogía

C es como el nivel de tolerancia del modelo. Un C alto dice: “no acepto ningún error”. Un C bajo dice: “acepto algunos errores si eso me da más margen general”.

El truco del kernel

¿Qué pasa si los datos no son linealmente separables en ninguna dimensión razonable?

Idea: mapear a mayor dimensión

Proyectar x a un espacio de alta dimensión $\phi(x)$ donde las clases sí sean separables.

El hiperplano en ese espacio corresponde a una frontera **no lineal** en el espacio original.

Truco del kernel

La formulación dual de SVM solo usa x_i a través de productos internos $\langle x_i, x_j \rangle$.

Si reemplazamos $\langle x_i, x_j \rangle$ por $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$, calculamos el producto interno en el espacio transformado **sin calcularlo explícitamente**.

Es como operar en una dimensión altísima sin pagar el costo computacional.

Kernels más usados

Kernel	Fórmula	Cuándo usarlo
Lineal	$x^\top x'$	Datos de alta dim. (texto)
Polinomial	$(\gamma x^\top x' + r)^d$	Interacciones polinomiales
RBF	$\exp(-\gamma \ x - x'\ ^2)$	Uso general, no lineal
Sigmoide	$\tanh(\gamma x^\top x' + r)$	Similar a redes neuronales

Kernel RBF: el parámetro γ

- γ pequeño: fronteras suaves, más regularización.
- γ grande: fronteras muy ajustadas al entrenamiento, riesgo de sobreajuste.

SVM multiclase y consideraciones prácticas

SVM es binaria por definición

Para $K > 2$ clases se usan estrategias de descomposición:

- **Uno vs. todos (OvR)**: un clasificador por clase.
- **Uno vs. uno (OvO)**: un clasificador por par de clases; predicción por votación.

Escala de los datos

SVM es sensible a la escala de las variables. Siempre estandarizar antes de entrenar. No hacerlo es uno de los errores más comunes en la práctica.

¿Cuándo usar SVM?

Funciona muy bien con pocos datos y alta dimensión (texto, imágenes pequeñas). Costoso para n muy grande (la solución requiere todos los vectores de soporte).

La idea detrás de KNN

Principio

Para clasificar un punto nuevo x_0 : encontrar sus k vecinos más cercanos en el conjunto de entrenamiento y asignar la clase más frecuente entre ellos.

Analogía cotidiana

“Dime con quién andas y te diré quién eres.”

KNN asume que los puntos cercanos en el espacio de features pertenecen a la misma clase. Si los k vecinos más cercanos son 4 de clase A y 1 de clase B, predecimos clase A.

No hay entrenamiento explícito: el modelo “aprende” guardando todos los datos. La predicción es el paso costoso.

Clasificador k -NN

$$\hat{y}(x_0) = \arg \max_c \sum_{i \in \mathcal{N}_k(x_0)} 1(y_i = c)$$

donde $\mathcal{N}_k(x_0)$ es el conjunto de los k índices con menor distancia a x_0 .

La distancia habitual es la **euclídea**:

$$d(x, x') = \sqrt{\sum_{j=1}^p (x_j - x'_j)^2}$$

Alternativas: Manhattan (L_1), Minkowski (L_p), Mahalanobis.

Elección de k : el hiperparámetro clave

k pequeño ($k = 1$)

Frontera muy irregular.

El modelo memoriza cada dato.

Alta varianza, bajo sesgo.

Como adivinar el partido de toda la escuela a partir del voto de una sola persona.

k grande

Frontera muy suave.

El modelo ignora detalles locales.

Bajo sesgo, alto sesgo global.

Como preguntarle a toda la Argentina para clasificar un barrio.

En la práctica

Se elige k por validación cruzada. Probar valores impares para evitar empates en clasificación binaria. Valores típicos: $k = 5, 7, 11$.

La maldición de la dimensionalidad

El problema

En espacios de alta dimensión (p grande), las distancias entre puntos tienden a homogeneizarse: todos los puntos quedan “igual de lejos”.

Un dato sorprendente

En $p = 1$ dimensión, el vecino más cercano está a distancia $\sim 1/n$ en promedio.

En $p = 100$ dimensiones, para tener el 1% de los datos “cerca”, necesitaría cubrir el 63% del rango de cada variable.

El espacio se vuelve enormemente vacío conforme p crece.

Consecuencia para KNN

El concepto de “vecino cercano” pierde significado en alta dimensión. KNN funciona bien con p pequeño (idealmente ≤ 20).

Consideraciones prácticas de KNN

Escala: obligatorio normalizar

Si $x_1 \in [0, 1]$ y $x_2 \in [0, 10.000]$, la distancia euclídea queda dominada por x_2 . x_1 queda prácticamente invisible para el modelo.

Siempre estandarizar o normalizar antes de KNN.

Costo computacional

- Entrenamiento: prácticamente nulo (solo guardar los datos).
- Predicción: $\mathcal{O}(n \cdot p)$ por cada punto a predecir.
- Con n grande, la predicción se vuelve muy lenta.

¿Cuándo usar KNN?

Pocos datos, dimensión baja, y cuando el modelo necesita ser simple y no importa el tiempo de predicción. También es útil como baseline.

SVM vs KNN: resumen comparativo

Aspecto	SVM	KNN
Tipo de frontera	Global (hiperplano + kernel)	Local (mayoría de vecinos)
Entrenamiento	Costoso (n grande)	Instantáneo
Predicción	Rápida	Lenta (n grande)
Escala	Obligatorio normalizar	Obligatorio normalizar
Alta dimensión	Bien (kernel lineal en texto)	Mal (maldición dim.)
Interpretación	Vectores de soporte	Directa pero local
Hiperparámetros	C , kernel, γ	k , métrica

- **SVM** busca el hiperplano de máximo margen entre clases.
- El **margen blando** (parámetro C) permite violaciones controladas.
- El **truco del kernel** extiende SVM a fronteras no lineales sin calcular explícitamente la transformación.
- **KNN** predice por mayoría entre los k vecinos más cercanos.
- Elegir k es el compromiso sesgo-varianza del método.
- La **maldición de la dimensionalidad** limita KNN en espacios de alta dimensión.
- **Ambos modelos requieren normalización previa.**

Próxima clase: Redes neuronales, métricas de evaluación, SMOTE y pipelines.